Shader Documentation

Setup Documentation

Create Material for My Custom Shader

1. Find shader naming `RealisticWater` on this path ` Assets/TechnicalArt-DwiCandraKirana_Shader-Test/Shader/RealisticWater.shadergraph`.
2. Left-click `RealisticWater.shadergraph` then click and find `Material`.
3. Set color on Surface Input component in Material water waves that you have been created.
4. Find path ` Assets/TechnicalArt- DwiCandraKirana_Shader-Test/Models`
5. Drag and drop `WaterPlane.max` to Hierarchy.
6. Set your Material RealisticWater created to WaterPlane on Hierarchy.
7. Finish

Open Scene to see my Custom Shader

1. Find this path `Assets/TechnicalArt- DwiCandraKirana_Shader-Test/Scene`
2. Open scene naming ` TechnicalArt-DwiCandraKirana_Shader-Test`

Introducing

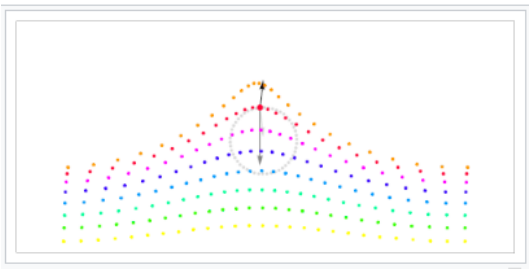


Figure 1. Gerstner Wave

A trochoidal wave or Gerstner wave is a solution to simulate ocean waves using shaders. This simulation forms waves by moving the vertices of a mesh both horizontally and vertically, creating a rolling and wavy visual effect.

$$X(a,b,t) = a + \frac{e^{kb}}{k}\sin(k(a+ct)),$$
$$Y(a,b,t) = b - \frac{e^{kb}}{k}\cos(k(a+ct)),$$

Figure 2. Formula

The vertical and horizontal wave motion is influenced by time, wavelength, speed, sharpness, and the use of sin (vertical) / cosine (horizontal) functions (Figure2).

Implementation

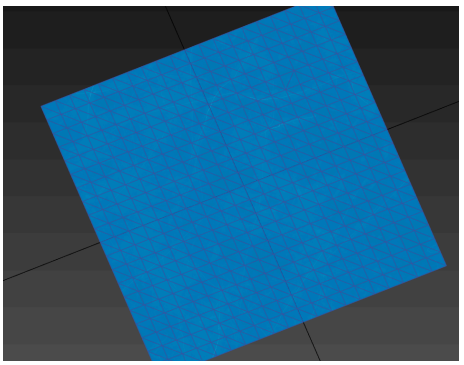


Figure 3. Water Plane Mesh

The first requirement is a mesh or water surface model consisting of many small triangles (triangular faces) that are interconnected, as shown in Figure 3.

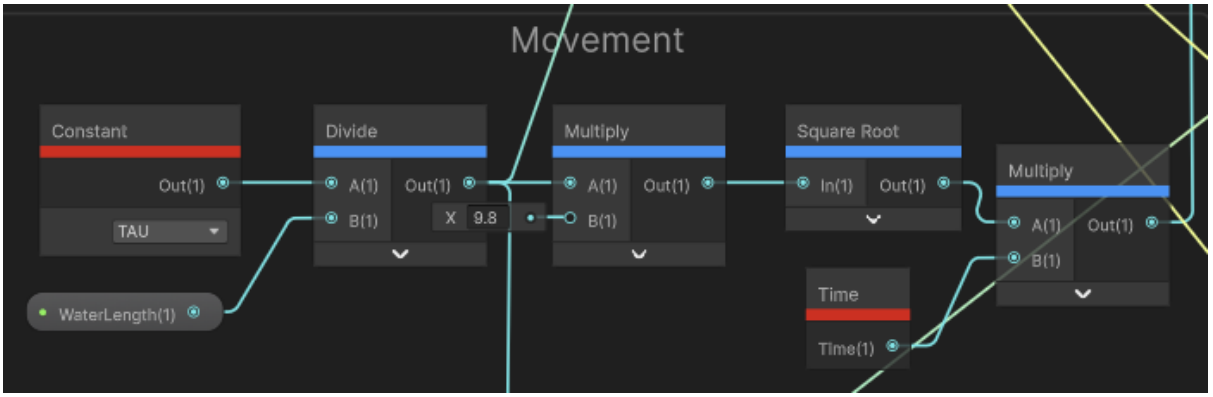


Figure 4. Variable

The first step is to create nodes that move the mesh both horizontally and vertically. Figure 4 performs calculations based on wavelength, default gravity (9.8), and time, to compute the angular frequency of the wave.

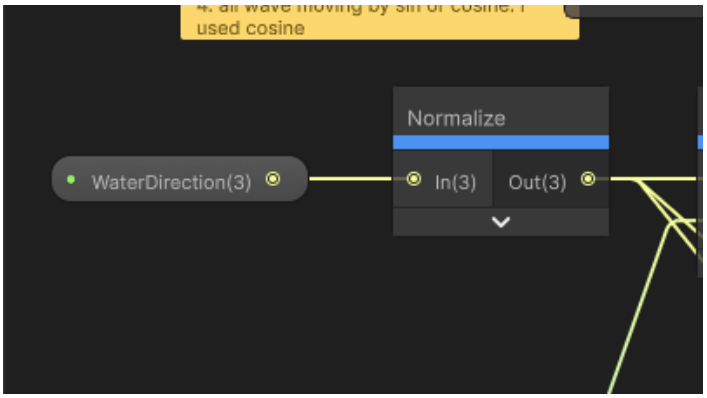


Figure 5. Wave Direction

Figure 5 is a node used to store the results of vertical and horizontal wave calculations before further processing.

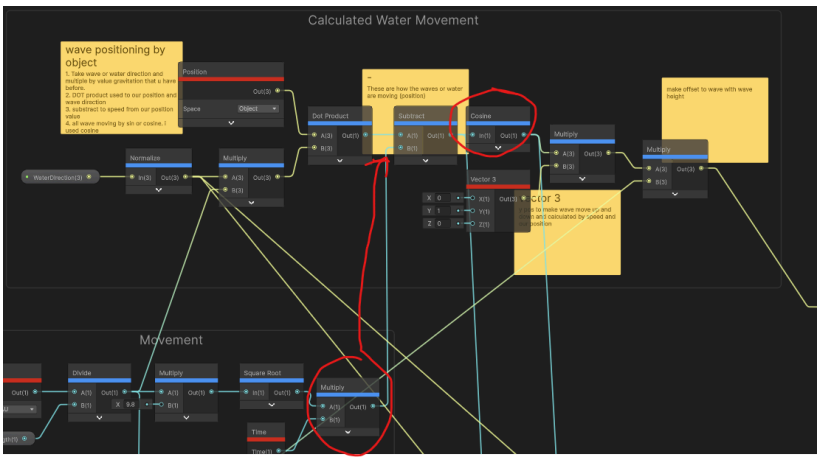


Figure 6. Cosine

Figure 6 calculates the horizontal movement of the mesh vertices by taking the object's position and determining its relation to the wave direction using the Dot Product. This result is then combined with wave speed from Figure 4 and passed through the cosine function to create the horizontal wave motion. At the end, the wave height is added as an offset to the mesh position.

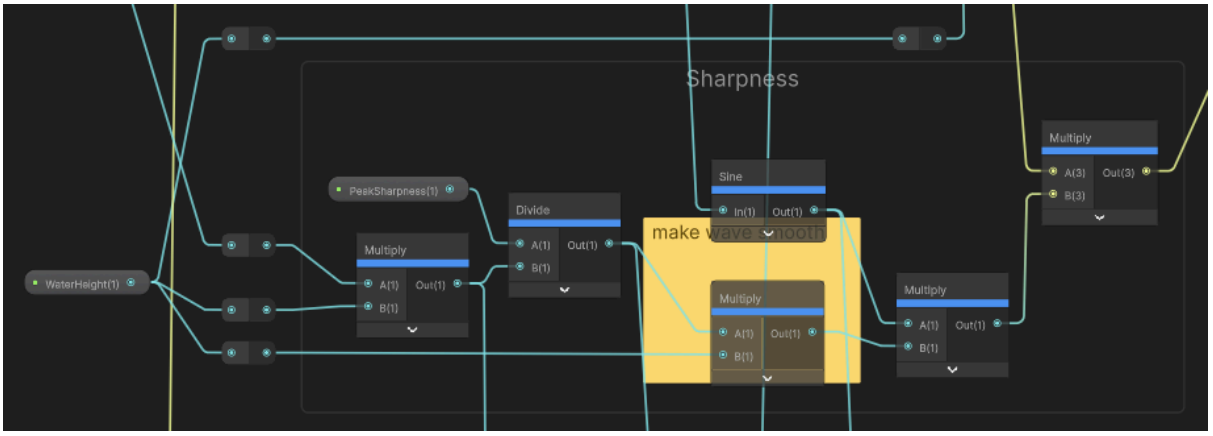


Figure 7. Sin (Vertical Wave)

To create a complete rolling wave motion, vertical wave movement is also needed. The vertical movement is calculated using the wave direction relative to the mesh object, then passed through the sine function. This movement is enhanced by wave sharpness to create a smoother wave effect.

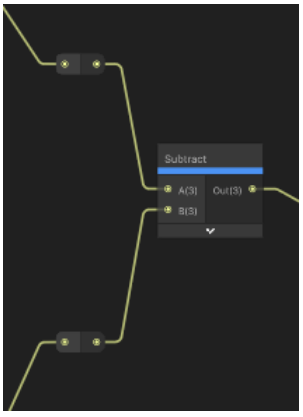


Figure 8. Calculate Wave

The combination of vertical and horizontal movement using Subtract is intended to produce a sharper rolling wave effect in one direction, giving the impression of a more dynamic water movement.

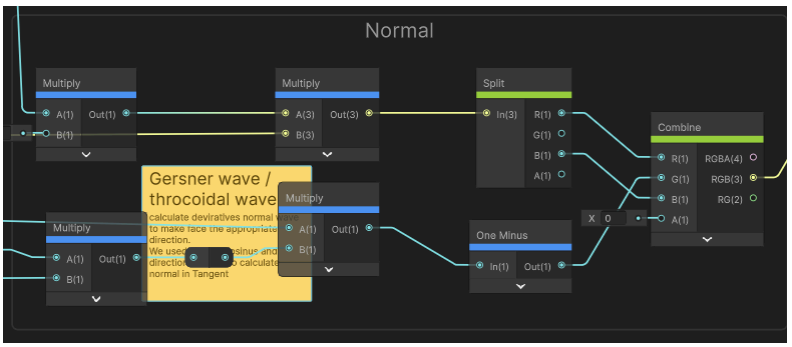


Figure 9. Devirates Wave

This calculates the derivatives of the wave position to obtain a normal vector that matches the direction of the water surface. By using the sin and cos functions and wave direction, normal texture can be calculated in tangent space to ensure light and shadow behave properly when the surface mesh deforms.

Why? Because when you move the vertices, the surface is no longer flat, and the default mesh normal becomes invalid; leading to incorrect lighting and reflection.

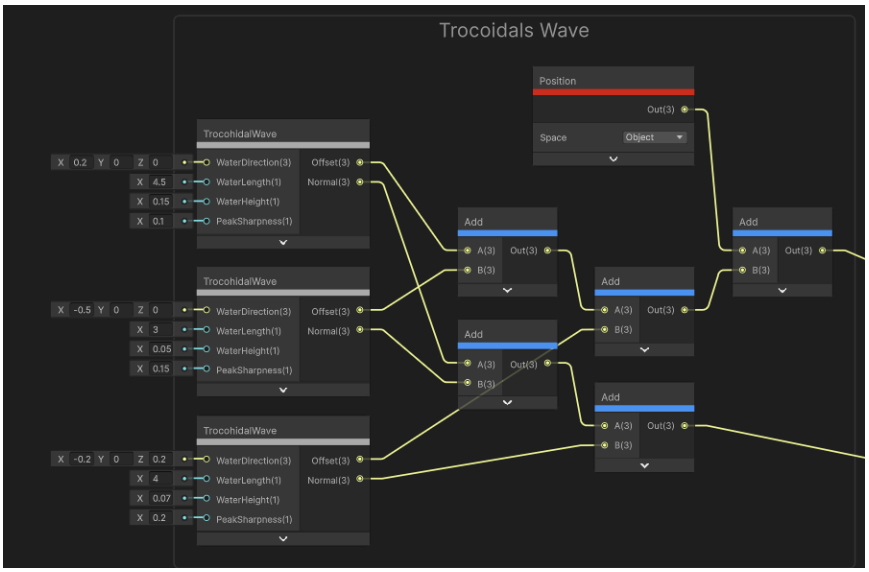


Figure 10. Gerstner or Trocoidals Wave

Figure 10 is combines three trochoidal wave functions to create dynamic vertex and normal deformations on the mesh. The combined position offset is used to manipulate the water surface, while the combined normals are used to produce lighting that aligns with the shape and direction of the wave.

UV and Ripple

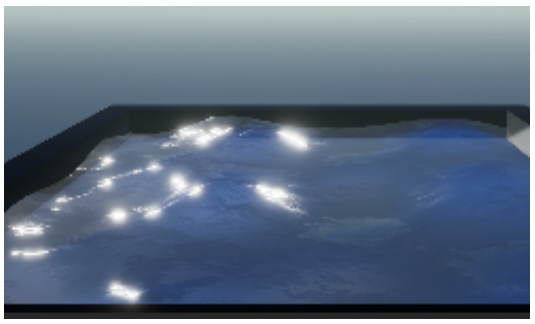


Figure 11. Ripple Surface

After applying wave motion to create wave shapes, UV setup is needed for the ripple effect to give a realistic impression of water from multiple viewing angles.

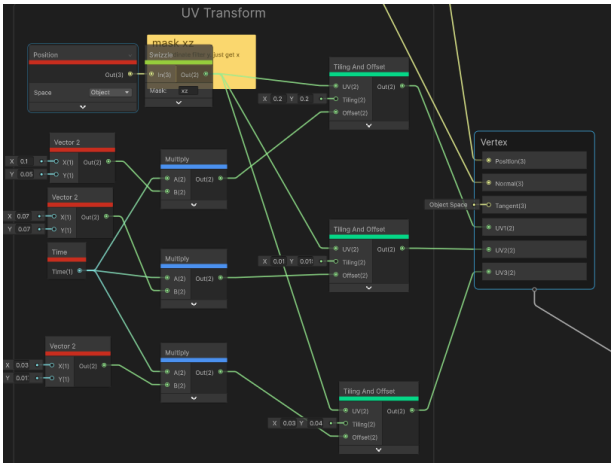


Figure 12. UV Transform

This section dynamically transforms UVs over time and direction, resulting in three pairs of UVs (UV1, UV2, UV3) used to animate the normal texture. UV direction only uses horizontal directions (X and Z) for scrolling.

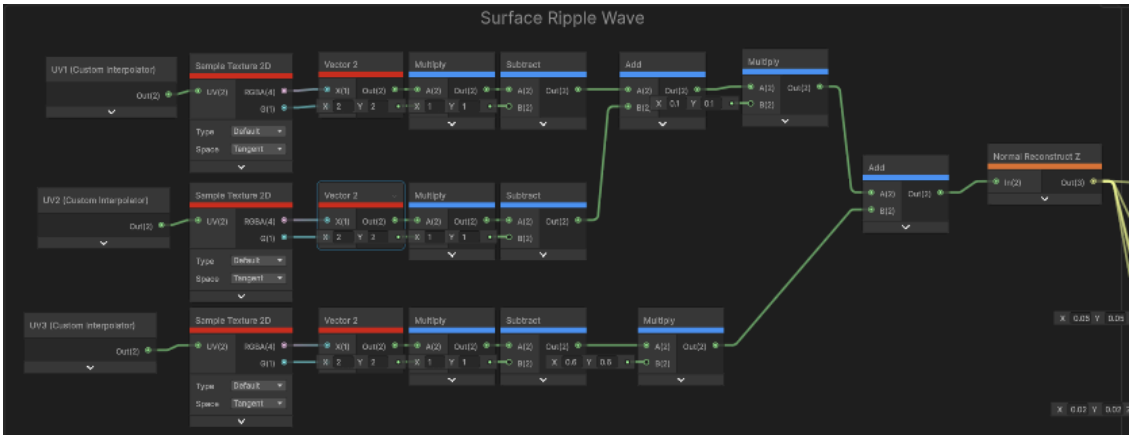


Figure 13. Surface Ripple Wave

To create specular effects, UV1–3 sample normal maps from three different directions and combine them to simulate surface reflection. The final normal is calculated in tangent space using the Normal Reconstruct Z node and applied in the fragment shader to produce refraction effects.

Transparency and Depth

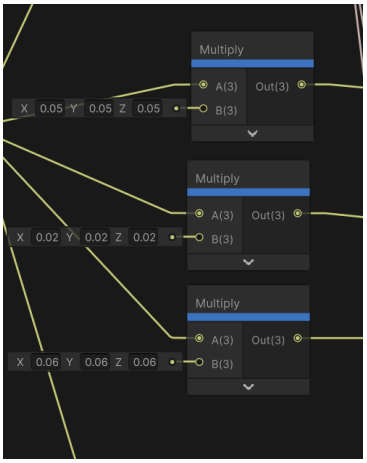


Figure 14. Transparency



Figure 15. Transparency

If using a box mesh, Figure 15's node can be used. It creates a deep water/sea floor transparency effect that follows the camera's position, where the sea floor or terrain beneath the water surface remains visible, and experiences color distortion based on screen position.

Figure 14 is used to reduce normal strength to break up individual normal textures.

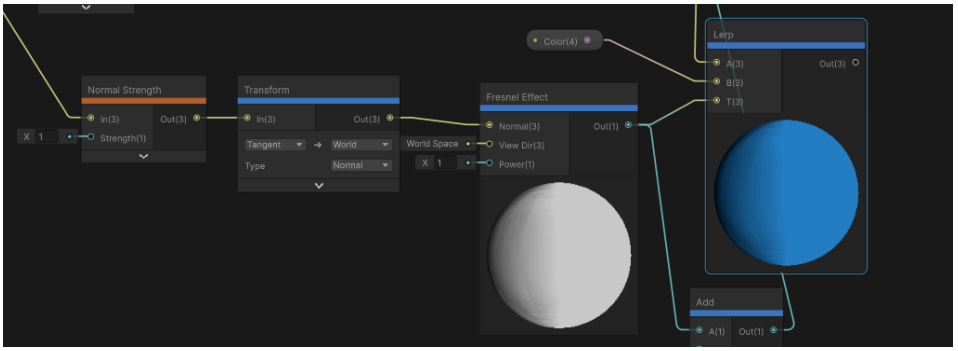


Figure 16. Opacity

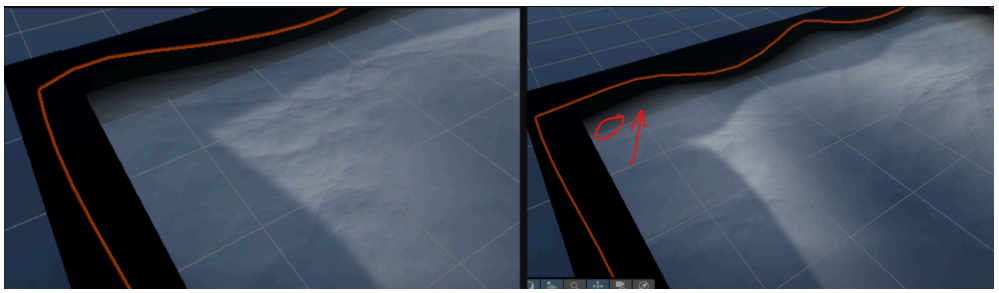


Figure 17. Freshnel Effect

Figure 16 applies the Fresnel effect to make water appear transparent when viewed from above, and more reflective (based on base color) when viewed from a low angle, as shown in Figure 17.

Final Effect: Glacier and Depth Water

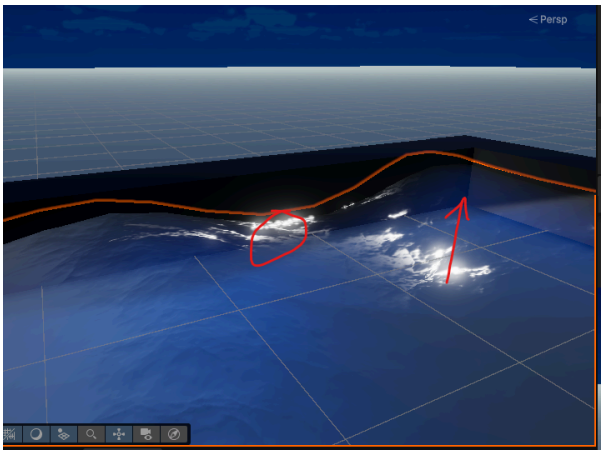


Figure 18. Glacier Effect

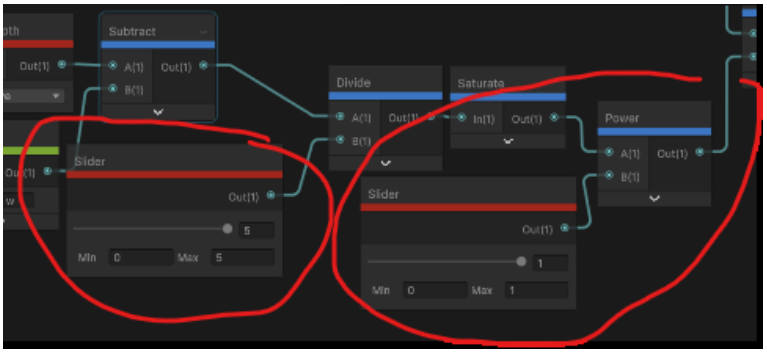


Figure 19. Depth Water

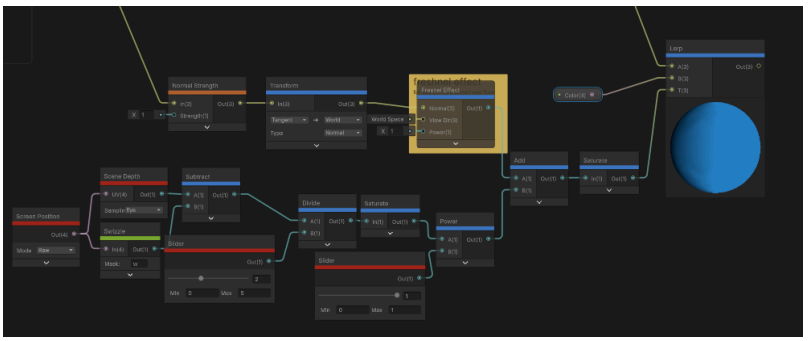


Figure 20 Blend Depth Water

This is an extension of the water effect by blending between Fresnel effect, base color, and refraction to simulate glacial water appearance. Figure 20 produces a glacier-like water effect by combining depth value and Fresnel effect. Depth adds a darker, denser color in the center. Fresnel Effect adds a brighter edge highlight depending on the viewing angle. Both are blended using Lerp, resulting in a natural color gradient like in Figure 18.